

Bismillahir rahmanur rahim
Ya allahu ya rahmanu ya rahim

Q1:

So when CI says "tests passed," what passed?

All the test functions you wrote ran without throwing assertion errors. That's the literal meaning. The CI runner:

1. Compiled your code
2. Ran project's test suite (e.g., `mvn test` or `npm test`)
3. Captured the exit code — `0` means success, anything else means failure
4. Reported pass/fail to GitLab

If we didn't write a test for a bug, CI won't catch it. CI is only as good as our test coverage. A green pipeline is *not* proof code works — it's proof code didn't break the things we remembered to check.

This is why engineers obsess about test quality, not just test count.

Q2: Difference between Local Development vs CI/CD — The Mental Model Shift

Local machine is a unique snowflake. We have Java 17.0.5, colleague has 17.0.8, the prod server has 17.0.2. You have Postgres 15, prod has Postgres 14. We have weird stuff in `/etc/hosts`. Our `JAVA_HOME` points somewhere specific.

This is the famous "works on my machine" problem. Code runs fine for us in local, breaks for everyone else, because everyone's environment is subtly different.

What happens in GitLab CI/CD

1. push code to GitLab
2. GitLab reads `.gitlab-ci.yml`
3. GitLab tells a **runner** (a server somewhere) to execute the pipeline
4. The runner spins up a **fresh, clean Docker container** — usually from an image like `maven:3.9-eclipse-temurin-17`
5. Inside that container, it clones repo, runs scripts (`mvn test`)
6. Container is destroyed when done

Key insight: **Every CI run starts from a clean, identical environment.** No leftover files. No "I installed that library last week and forgot." It's deterministic.

[Local laptop]

1. Edit UserService.java in IntelliJ
2. Run unit tests locally (Cmd+Shift+F10) — they pass
3. Run the app, hit the API with Postman — works
4. git add . && git commit -m "Add user search"
5. git push origin feature/user-search



[GitLab.com]

6. GitLab sees the push, reads .gitlab-ci.yml
7. Triggers a pipeline



[GitLab Runner — a clean Linux VM/container somewhere]

8. Pulls maven:3.9-eclipse-temurin-17 Docker image
9. Clones our repo into the container
10. Runs: mvn clean install
11. Runs: mvn test
12. If anything fails → ❌ pipeline fails, we get an email
13. If everything passes → ✅ artifacts are saved (the .jar file)



[Next stage: build Docker image]

14. Takes the .jar, builds a Docker image using Dockerfile
15. Pushes image to a registry (GitLab's, or AWS ECR, etc.)



[Next stage: deploy]

16. Tells the production server (or Kubernetes) to pull the new image and restart with it



[Production]

17. Users now use the new code

Era 1: The Dark Ages (1990s–early 2000s)

- Developers wrote code, zipped it up, emailed it to "Ops" team
- Ops manually copied files to servers via FTP
- Releases happened quarterly, took entire weekends
- Nobody knew what version was running where
- Rollbacks meant restoring from tape backups
- "Works on my machine" was a daily occurrence

Era 2: Scripts and SSH (mid 2000s)

- Smart developers wrote bash/Perl scripts to deploy
- SSH into server, run `deploy.sh`, hope for the best
- Better, but scripts were undocumented, fragile, machine-specific
- New team member? Good luck running the deploy script

Era 3: Configuration Management (late 2000s)

- Tools like Puppet, Chef, Ansible emerged
- "Infrastructure as code" — describe servers in config files
- Reproducible server setups
- But still: code ran on a host OS, with all its quirks

Era 4: Virtual Machines (2008-ish)

- VMware, VirtualBox — package a whole OS as a file
- Solved environment problems but VMs are HUGE (gigabytes), slow to start
- Vagrant tried to make this dev-friendly

Era 5: Docker arrives (2013)

- Containers: lightweight, fast, portable
- Suddenly "package app and environment together" became practical
- Image is megabytes, starts in seconds
- This was a genuine revolution

Era 6: CI/CD goes mainstream (2015+)

- Jenkins existed since 2005 but was clunky (XML configs, plugins hell)
- Travis CI, CircleCI introduced YAML-based config in repos
- GitLab CI (2015) integrated CI directly into the Git platform — game changer
- GitHub Actions (2019) followed the same pattern

Era 7: Kubernetes & GitOps (2018+)

- Kubernetes orchestrates containers at scale

- **GitOps: Git is the source of truth for infrastructure too**
- **Argo CD, Flux: deployments triggered by Git commits to infra repos**

Why we changed each thing

What we left behind	Why
Manual deploys	Error-prone, slow, not reproducible
Custom bash scripts	Undocumented, fragile, knowledge silos
VMs	Too heavy, slow to spin up, big files
Jenkins XML configs	Not in version control, hard to review
Snowflake servers	Drift between environments caused bugs
	Slow feedback, big risky deploys

The modern principles that emerged:

Everything in Git — code, infra, CI config, even secrets (encrypted)

Immutable artifacts — build once, deploy the same artifact everywhere

Fast feedback — failures detected in minutes, not weeks

Reproducibility — anyone can rebuild anything from source

Small, frequent deploys — less risk per deploy, easier rollback

The problem Docker solves

Imagine deploying your Spring Boot app. The server needs:

- Java 17 installed
- Specific environment variables set
- The right OS libraries
- `.jar` file in the right place
- A specific user with the right permissions

If you set this up manually on every server, you'll make mistakes. If you use 10 servers, you'll have 10 slightly different setups. Hell.

What Docker does

Docker lets package **app + its entire environment** into one portable file called an **image**. That image runs identically anywhere: your laptop, CI, AWS, your colleague's machine.

Think of it like this:

- **Image** = a frozen snapshot of a complete, ready-to-run mini-computer (immutable, like a class)
- **Container** = a running instance of that image (like an object)

```
# — Stage 1: Build the JAR —
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline    # cache dependencies
COPY src ./src
RUN mvn clean package -DskipTests # build the JAR
```

```
# — Stage 2: Run the JAR —
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
build-image:
  stage: package
  image: docker:24
  services:
    - docker:24-dind    # Docker-in-Docker
  script:
    - docker build -t registry.gitlab.com/myorg/myapp:$CI_COMMIT_SHA .
    - docker push registry.gitlab.com/myorg/myapp:$CI_COMMIT_SHA
```

The flow:

Code → CI tests it → CI builds Docker image → Image pushed to registry → Production server pulls image → Container starts → Users hit your API

```
stages:
  - build      # compile and test
  - package    # build Docker image
  - deploy-staging
  - deploy-prod
```

```

variables:
  MAVEN_OPTS: "-Dmaven.repo.local=.m2/repository"
  IMAGE_NAME: $CI_REGISTRY_IMAGE
  IMAGE_TAG: $CI_COMMIT_SHA    # unique per commit

# Cache Maven deps so we don't re-download every run
cache:
  paths:
    - .m2/repository/

# ——— Build & Test ———
build-and-test:
  stage: build
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn clean verify    # compiles, runs unit + integration tests
  artifacts:
    paths:
      - target/*.jar
    reports:
      junit: target/surefire-reports/TEST-*.xml    # GitLab shows test
results in UI
      expire_in: 1 week

# ——— Build Docker image ———
build-image:
  stage: package
  image: docker:24
  services:
    - docker:24-dind
  needs:
    - build-and-test    # don't run until tests pass
  script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD
$CI_REGISTRY
    - docker build -t $IMAGE_NAME:$IMAGE_TAG -t $IMAGE_NAME:latest .
    - docker push $IMAGE_NAME:$IMAGE_TAG
    - docker push $IMAGE_NAME:latest
  rules:
    - if: $CI_COMMIT_BRANCH == "main" || $CI_COMMIT_BRANCH == "develop"

# ——— Deploy to staging (auto on develop) ———

```

```

deploy-staging:
  stage: deploy-staging
  image: alpine:latest
  script:
    - apk add --no-cache curl
    - curl -X POST $STAGING_DEPLOY_WEBHOOK -d
      "image=$IMAGE_NAME:$IMAGE_TAG"
  environment:
    name: staging
    url: https://staging.myapp.com
  rules:
    - if: $CI_COMMIT_BRANCH == "develop"

# — Deploy to prod (manual on main) —
deploy-prod:
  stage: deploy-prod
  image: alpine:latest
  script:
    - apk add --no-cache curl
    - curl -X POST $PROD_DEPLOY_WEBHOOK -d "image=$IMAGE_NAME:$IMAGE_TAG"
  environment:
    name: production
    url: https://myapp.com
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual    # human must click "Deploy"

```

The whole story this file tells:

1. Push to any branch → tests run
2. Push to **develop** or **main** → also build & push Docker image
3. Push to **develop** → auto-deploy to staging
4. Push to **main** → image is ready, but deploy needs a human click